

FDP: The Data Placement Promise of Modern NVMe SSDs [Experiment, Analysis & Benchmark]

Sijie Lan
Pennsylvania State University
sml6844@psu.edu

Hui Qi
Samsung
hui81.qi@samsung.com

Xing He
Samsung
xing1.he@samsung.com

Mahmut Kandemir
Pennsylvania State University
mtk2@psu.edu

Javier González
Samsung
javier.gonz@samsung.com

Vivek Shah
Samsung
vi.shah@samsung.com

ABSTRACT

NVMe SSDs are now widely deployed as the storage tier in data centers. As SSDs have evolved over the past decade, the community has continued to debate the interfaces they expose and how operating systems and storage systems should exploit them. The NVMe Flexible Data Placement (FDP) proposal is the latest point in this design space. FDP introduces an interface based on Reclaim Units that enables explicit data placement to reduce device write amplification without the software engineering costs of sequential-write constraints and host garbage collection. FDP-enabled SSDs are emerging in commercial products and early data center deployments. Their compatibility with conventional block I/O allows existing applications to run unchanged, allowing a frictionless adoption in industry.

This paper presents an experimental evaluation of FDP SSDs to characterize their data placement guarantees over the raw device interface. We then revisit two widely deployed and distinct open-source storage systems, MySQL and RocksDB, and examine whether lifetime-based data separation and distinct write patterns built into their architectures can be mapped onto FDP SSDs without invasive changes. Our evaluation shows end-to-end WAF reductions at higher device utilization, along with QoS and throughput improvements under synthetic and real-world workloads. These results demonstrate that FDP provides a practical and deployable cross-layer mechanism for data placement with open-source ecosystem support on Linux. They also highlight why FDP SSDs are gaining traction in industry.

PVLDB Reference Format:

Sijie Lan, Hui Qi, Xing He, Mahmut Kandemir, Javier González, and Vivek Shah. FDP: The Data Placement Promise of Modern NVMe SSDs [Experiment, Analysis & Benchmark]. PVLDB, 14(1): XXX-XXX, 2020. doi:XX.XX/XXX.XX

PVLDB Artifact Availability:

The source code, data, and/or other artifacts have been made available at https://github.com/sijielan/FDP_Benchmarking.git.

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit <https://creativecommons.org/licenses/by-nc-nd/4.0/> to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 14, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

1 INTRODUCTION

NVMe SSDs have become the de facto persistent storage medium in modern datacenters due to their performance-cost trade-offs compared to DRAM and hard disk drives [50]. However, the conventional block interface [9] obscures internal flash management, preventing applications from conveying data placement information that is critical for reducing device write amplification factor (WAF) and improving device lifetime. Over the past decade, both academia and industry have explored alternative interfaces [50], ranging from fully host-managed Open Channel SSDs [15] to partial host-managed designs such as Zoned Namespaces (ZNS) [14], which improve data placement at the cost of software complexity.

Flexible Data Placement (FDP) SSDs [6, 10, 44] represent a new point in this design space based on lessons learned from ZNS. FDP exposes placement primitives that allow hosts to group writes with similar lifetimes while retaining the conventional block interface. This interface compatibility enables incremental deployment where FDP devices can replace existing SSDs without disrupting FDP-unaware applications, while allowing FDP-aware systems to exploit data placement when beneficial. Vendors such as Samsung, Kioxia, and DapuStor are targeting FDP SSDs for enterprise datacenters to reduce WAF, increase utilization, and extend device lifetime in both single-tenant and multi-tenant settings that are increasingly appearing in datacenter deployments [19, 20, 32, 42, 43].

Stable Linux kernel support for NVMe I/O Passthru [29] makes FDP practical for widely deployed systems via the character device using userspace NVMe commands without waiting for filesystem support. Early work has shown that modest application changes can exploit FDP to separate hot and cold data in Flash cache architectures, significantly reducing WAF and extending SSD lifetime in multiple production workloads [10]. The work has already been merged into CacheLib [13] and deployed in production. These results raise the question, "*Can other system classes exploit architectural data separation to reduce device write amplification and increase SSD lifetime using FDP interfaces exposed by the Linux kernel?*".

I/O Passthru is a natural fit for CacheLib because it interacts with the SSD over the block interface. However, most systems rely on a filesystem interface and designing a filesystem wrapper for them is invasive and creates friction for production deployment. Early experimental Linux kernel support for FDP through filesystems using lifetime hints [7, 8] provides an opportunity to use FDP primitives at a lower software engineering cost. *This paper explores the costs and benefits of this opportunity for two widely deployed and complementary classes of data systems: LSM-tree-based key-value stores*

(e.g., RocksDB [22]) and B-tree-based relational database systems (e.g., MySQL [16]). These systems have architectural components with distinct data lifetimes and write patterns but the conventional storage interface prevents the underlying device from exploiting it.

LSM-tree-based key-value stores incur high device WAF under write-intensive workloads and background compaction [36, 38]. Although LSM-tree-based architectures separate files into levels with different lifetimes, this separation is lost at the storage interface. Moreover, LSM-tree-based systems perform flash-friendly out-of-place writes during compaction by deleting files and appending data to new files, which limits their consumption of the SSD’s over-provisioned (OP) erase blocks. Our analysis shows that files within each level exhibit similar lifetimes, with lower levels being significantly hotter and smaller than higher levels. By propagating this architectural separation through FDP placement hints across both experimental filesystem and stable I/O Passthru interfaces (filesystem bypass) in the Linux kernel, we reduce RocksDB’s device WAF to approximately 1 at 90% device utilization, nearly eliminating compaction-induced device write amplification.

In contrast, B-tree-based relational database systems [25], such as MySQL [16], exhibit a mix of in-place and out-of-place updates across write-ahead logs, buffer managers, and double-write buffers. Our analysis shows that this mixture, together with random and sequential write patterns, leads to high device write amplification at increased device utilization. By separating these components using filesystem-level FDP hints with minimal code changes, we reduce device write amplification to ~1.2 at 80% device utilization and extend SSD lifetime without building a specialized DBMS-managed translation layer with exclusive SSD ownership [33]. Our work shows the promise of data separation alone without the costs of garbage collection on FDP SSDs.

First-generation enterprise FDP SSDs are becoming available from vendors for data center deployments, raising questions among industry practitioners and researchers about their isolation and QoS behavior. Prior work has studied FDP through emulation [44], but it has primarily focused on reproducing existing workloads [10] and the impact of emulation. In our characterization, we systematically evaluate the current generation of enterprise FDP SSDs [41], ranging from 2 TB to 8 TB through the raw device interface. We quantify the limits of their isolation and QoS behavior and identify the factors that affect them under concurrent, write-intensive workloads. We hope this characterization provides a foundation for designing system abstractions for filesystems, storage systems, and database systems, and can guide further work in this area. Our end-to-end data placement approach for widely deployed open-source systems, RocksDB and MySQL, uses this characterization to exploit application-level data separation with device behavior. This approach reduces device WAF and extends SSD lifetime in single-tenant and multi-tenant settings along with improvements in application-level metrics of throughput and latency.

In this work we make the following contributions:

1. Enterprise FDP SSD Characterization. We analyze the isolation and QoS behavior of commercially available enterprise FDP SSDs for data centers and quantify the factors that affect them under concurrent, write-intensive workloads (§ 3).

2. RocksDB Data Placement. We show that the LSM-level architecture and compaction process of key-value stores provide data

separation that maps naturally to FDP data placement. Using the filesystem interface, this mapping reduces device WAF to near-ideal levels with changes to only 13 lines of code. Bypassing the filesystem through the I/O Passthru interface further improves tail latency and application-level write amplification under intense write pressure (§ 4).

3. MySQL Data Placement. We show that RDBMS components such as logging, buffer management, and double-write buffering exhibit distinct write patterns and lifetimes that can be separated using FDP. Using the filesystem hinting interface, this separation reduces device WAF, supports higher device utilization, and improves application throughput (§ 5).

4. End-to-End Evaluation. We perform a comprehensive evaluation of the benefits of data separation across RocksDB, MySQL, emerging and stable Linux kernel interfaces, and FDP SSDs using multiple synthetic and real-world benchmarks in single-tenant and multi-tenant settings (§ 6).

2 BACKGROUND AND MOTIVATION

2.1 FDP In a Nutshell

Flexible Data Placement (TP 4146) was fully ratified by the NVMe board in 2023 [6]. It originated from a joint proposal by Google and Meta that merged their SmartFTL [40] and Direct Placement Mode designs. FDP extends the multi-stream SSD proposal [30] by providing explicit data placement guarantees on SSDs while avoiding the garbage collection responsibilities imposed by ZNS [14]. **The proposal is fully backward-compatible, i.e., an FDP SSD can operate as a conventional SSD and does not introduce new NVMe commands.**

An FDP SSD organizes NAND flash blocks into a collection of Reclaim Units (RUs) within a reclaim group (RG). The host is given a set of reclaim unit handles (RUHs), which act as placement tags for NVMe write commands. The size of the reclaim unit is vendor-defined and may span multiple erase blocks. Emerging FDP SSDs use superblock-sized reclaim units to increase parallelism and reduce firmware indexing overheads. The SSD retains full internal control over the mapping between RUHs and reclaim units. The read commands operate without any modification. *If a write command does not specify a RUH, the device applies a default reclaim unit handle.* Standard NVMe admin commands allow the host to discover the FDP topology, including reclaim groups, reclaim unit size, and the number of available reclaim unit handles.

Unlike ZNS, logical block addresses are not statically bound to particular RUHs. Instead, they are *dynamically* assigned as the host issues RUH tagged NVMe write commands. While FDP removes host control over garbage collection, it provides a set of event types that capture important garbage collection-related information reported by the SSD controller. These events are recorded in an NVMe log page and can be queried by the host to evaluate placement effectiveness and integrate into feedback driven placement policies. For details on current generation FDP SSD architecture and programming model, we refer readers to whitepapers [42, 43], educational videos [11], and research papers [10, 50].

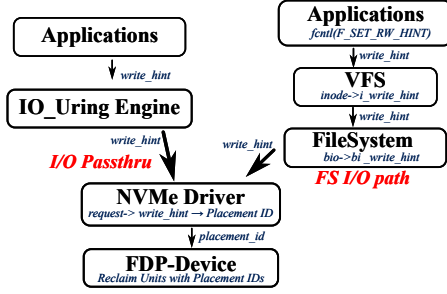


Figure 1: FDP write hint propagation through I/O Passthru and filesystem paths in the Linux kernel.

2.2 FDP Linux Support

The FDP proposal is being driven by hyperscalers, in contrast to earlier proposals that were primarily led by SSD vendors. As a result, supporting FDP within the Linux open-source ecosystem has been a priority, drawing on lessons from prior efforts, especially the Multi-Stream proposal [30]. Applications can send data placement hints from userspace in Linux through the following mechanisms (see Figure 1).

1. Using the raw character device abstraction and I/O Passthru.

An application can access an FDP SSD through a *raw character device* registered by the kernel. As shown in the "I/O Passthru" path in Figure 1, the application constructs NVMe write commands with RUH hints using the `NVME_RW_DTYPE_DPLCMT` directive type and submits them to the Linux kernel via the `io_uring_cmd` interface [29], which bypasses the Virtual File System (VFS) and block layers and forwards the command directly to the Linux NVMe driver and the SSD. This interface provides applications with full control over specifying RUH hints on a per-LBA basis. I/O Passthru support has been available in the Linux kernel since version 5.19. This mechanism was used to integrate FDP support into CacheLib over the raw character device [10].

2. Using the filesystem and per-file lifetime hints. To support applications that use the Linux filesystem abstraction rather than raw device access, a recent patchset has been proposed [4, 5, 7, 8]. The patchset enables propagation of existing per-file lifetime hints in the Linux VFS layer to disjoint RUHs. Applications can pass lifetime hints via the `fcntl` system call using one of the available values, namely, `SHORT`, `MEDIUM`, `LONG`, `EXTREME`, `NOT_SET`, `NONE`. The hint is stored in the inode, and all I/O to the corresponding file is tagged with this hint. As shown in the "FS I/O path" in Figure 1, the inode-level hint is propagated to the NVMe driver, which maps the lifetime hint to a disjoint RUH using the `NVME_RW_DTYPE_DPLCMT` directive type. If the SSD supports fewer RUHs than the number of hints, the default RUH is used. Since this mechanism operates at the VFS and NVMe driver layers, it becomes automatically available across all Linux filesystems. Although this approach is highly convenient, it limits the granularity of data placement to the file level rather than individual LBAs. The mechanism is intended to allow applications to adopt FDP SSDs seamlessly through filesystem interfaces while still enabling deathtime-based grouping.

2.3 The Flexible Data Placement Opportunity

In this paper, we evaluate the potential of architectural data separation through FDP using two widely deployed storage systems with

distinct architectures:

1. **LSM tree based key-value stores** (§ 2.3.1)
2. **B-tree based relational database systems** (§ 2.3.2)

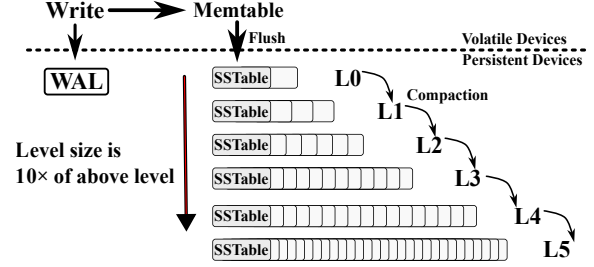


Figure 2: RocksDB Architecture Overview.

2.3.1 Key-Value Stores using LSM tree-based architectures (RocksDB).

Key-value stores such as RocksDB [22] use log-structured merge trees (LSM trees) [36, 38]. As shown in Figure 2, each write is appended to a *write-ahead log* (WAL) and inserted into an in-memory *MemTable*. When the MemTable fills, RocksDB flushes it as an immutable sorted string table (SST) file to level L0. Background *compaction* merges SST files and propagates their contents to deeper levels (L1...Ln), discarding obsolete entries when possible. WAL and SST files are written sequentially and obsolete files are later deleted. Although this out-of-place write pattern is conceptually SSD friendly, the lack of data separation at the interface results in data mixing at the SSD level between the different LSM levels and the WAL. Moreover, under leveled compaction the level size increases geometrically (size multiplier) across levels. Lower-numbered levels are therefore smaller and experience higher file deletion and creation rates than deeper levels.

Moreover, in leveled compaction strategies, the sizes of levels vary exponentially. Consequently, level L0 is significantly smaller in size and more frequently updated than level L1. This property holds throughout the level structures.

SST Lifetime Analysis. To test whether SST files can be grouped by expected deletion (death) time, we measured file lifetimes across LSM-tree levels. We instrumented RocksDB to record the creation and deletion timestamps of each SST file during workload execution. The interval between these events gives the observed lifetime of each deleted file. Across multiple five-hour runs of synthetic, compaction-intensive write-heavy workloads, files in L0-L3 had short, relatively uniform lifetimes. Most persisted for seconds to minutes before being selected for compaction. File lifetimes diverged from L4 onward. Files in L4 persisted for hundreds to tens of thousands of seconds, while many files in L5 remained present at the end of each run. These results indicate a marked placement boundary at L4, suggesting that files in L0-L3 and files in L4-L5 should be assigned to separate placement groups. This separation boundary is a function of the architecture and not the specific workload.

In this work, we study whether separating the WAL and LSM-tree levels through the filesystem and I/O Passthru interfaces is sufficient to reduce device WAF at high utilization on FDP SSDs.

2.3.2 B-Tree based Relational Database Systems (MySQL). Relational database systems such as MySQL [16] rely on B-tree (and its

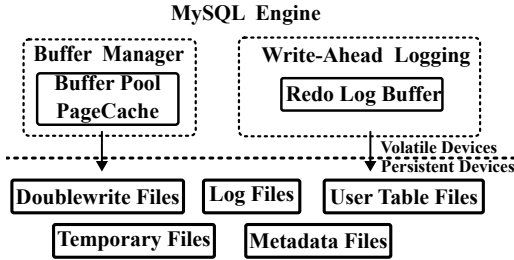


Figure 3: Data classes written by the MySQL/InnoDB engine. All writes are staged through the in-memory buffer pool and then land on storage as user tables, metadata, the double-write buffer, redo logs, and temporary files.

variants) to organize table data and indexes into pages managed by a buffer pool [25]. While updates modify pages in memory, dirty pages are later flushed to their locations in-place in the data files, resulting in random writes on SSDs. To support crash recovery, MySQL’s InnoDB storage engine records changes in a sequentially written redo log analogous to a write-ahead log in a key-value store. InnoDB writes pages being flushed to a doublewrite buffer before writing them to their final locations in the data files. This protects against torn page writes during a crash.

Percona Server for MySQL is an open-source, performance-enhanced drop-in replacement for MySQL Community Edition [27] that is widely deployed in high-throughput transactional environments. As shown in Figure 3, it uses InnoDB as its default storage engine, whose storage files fall into five categories: (1) user table files, (2) log files, (3) temporary table files, (4) doublewrite buffer files, and (5) metadata files [18]. These categories exhibit distinct lifetime and write-pattern characteristics, making them natural candidates for differentiated data placement. User table files are persistent and receive predominantly random in-place updates, whereas redo log files are sequentially written and cyclically reused. Temporary table files are short-lived, while doublewrite buffer files are fixed in size and cyclically overwritten at high frequency. Metadata files are generally smaller and exhibit different update characteristics from user data. Our empirical characterization reveals that user table and doublewrite buffer files dominate the write traffic, accounting for 44.9% and 43.7% of host writes, respectively. In contrast, metadata and log files account for 7.0% and 4.3%, respectively, while temporary files contribute less than 0.1%.

Concurrent updates from these storage components lead to mixing of data with different lifetimes in the same SSD erase blocks causing increased device-level write amplification. *In this work, we study whether MySQL’s existing separation of storage files can be used to assign distinct lifetime hints through the filesystem interface, reducing device-level write amplification on FDP SSDs at high utilization.* The required data placement changes comprise only 31 lines of code across the source files.

2.3.3 Why is SSD write amplification important now? The lifetime or endurance of a NAND Flash-based SSD is largely determined by its *device write amplification factor (WAF)* due to the limited number of program/erase cycles and log-structured overwrite nature of Flash cells [9, 47, 50]. Enterprise SSD deployments in datacenters are actively provisioned and monitored so that the

device WAF remains close to ~1. Even modest increases in WAF can significantly reduce device lifetime and quality of service (QoS), directly translating into higher infrastructure costs at scale. Recent SSD supply constraints and rising NAND Flash costs have forced hyperscalers to maximize usable capacity, making capacity overprovisioning an unattractive mechanism for controlling WAF [28, 46]. Moreover, lowering WAF also improves datacenter sustainability [1–3, 37, 45] and it also correlates with *performance stability and SSD power consumption*. Excessive garbage collection not only wears out flash cells and consumes power, but also increases latency variability and contributes to tail latency spikes which are particularly harmful for latency-sensitive services [35].

3 THE FDP PROMISE OF THE RAW DEVICE

In this section, we evaluate the isolation behavior of FDP SSDs and their impact on device WAF, I/O throughput (IOPS), and latency under heavy concurrent write patterns that lead to high device WAF at full device utilization. This characterization establishes the potential benefits and limitations of separating data by lifetime at the application layer.

Tools and Setup. We use FIO (v3.35) [12] with the `io_uring_cmd` NVMe engine via `libnvme v1.16.1` on Linux kernel 6.9, which supports I/O Passthru to generate synthetic write workloads. Table 1 summarizes the microbenchmark configurations. We use the character device in these experiments to minimize any interference effects of the Linux kernel and its I/O stack. Our experiments are conducted on three PM9D3a FDP SSDs [41] with capacities of 1.88 TB, 3.84 TB, and 7.68 TB, each exposing 8 RUHs with reclaim unit sizes of approximately 6.44 GB, 13.08 GB and 17.54 GB, respectively. Due to space constraints, we present results for the 3.84 TB device in the main paper and provide a comparison across device capacities in § A.3 (supplementary material). We obtain device WAF using vendor-specific logs for accuracy¹. *All workloads are evaluated on the same FDP device in two modes, (1) with FDP enabled (FDP), and (2) with FDP disabled for the conventional NVMe SSD baseline (Conv).* In the FDP mode, workloads are assigned to a RUH while in the Conv mode they are not.

Our experiments address questions raised by datacenter operators about FDP SSD behavior under common workloads and deployment conditions:

1. How effective is RUH isolation under concurrent sequential workloads (§ 3.1)?
2. How effective is RUH isolation when RUHs are shared between concurrent writers (§ 3.2)?
3. How effective is RUH isolation under a mix of concurrent sequential and random workloads (§ 3.3)?
4. How effective is RUH isolation in the presence of mixed hot and cold data (§ 3.4)?
5. How effective is RUH isolation when concurrent workloads have mixed skewed and uniform access patterns (§ 3.5)?

3.1 RUH Isolation With Sequential Workloads

Setup. To quantify the best-case isolation benefits of data placement on FDP, we run eight FIO sequential write workloads over

¹WAF can also be measured using the NVMe OCP commands.

Table 1: FIO microbenchmark configurations.

Experiment	Jobs	Access pattern	Block size	Run length / stop condition
§3.1 Sequential (uneven / even)	8	Sequential write; ranges 2–23 % / 12 % each	128 KiB	8 h (time-based)
§3.2 Shared RUHs	8–64	Sequential write; 1, 2, 4, 8 jobs per RUH	128 KiB	8 h (time-based)
§3.3 Varying randomness	8	8S → 8R; smallest ranges become random	128 KiB	8 h (time-based)
§3.4 Hot/cold prefill	1	Random write after 50–99 % cold prefill	4 KiB	10× device capacity written
§3.5 Skew, single	1	Zipfian random write (iolog), $z = 0-1.4$	16 KiB	10× device capacity written
§3.5 Skew, co-running	2	Uniform + Zipfian random write, 3 LBA splits	4 KiB	8 h (time-based)

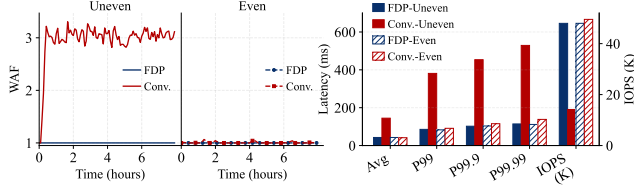


Figure 4: FDP vs. Conv. SSD WAF, latency, and IOPS under eight sequential streams with even and uneven partitioning.

disjoint LBA ranges, modeling a perfectly partitioned workload stressing the SSD while maintaining a constant invalidation rate of LBAs. In the FDP mode, each workload is pinned to a distinct RUH. Each workload writes sequentially within its assigned range and wraps around upon reaching the end. Each experiment runs for 8 hours to evaluate long-term behavior and resilience. We partition the device using two policies:

- (1) **Even Partitioning.** The device is divided into eight equal-sized, non-overlapping regions, each assigned to one write stream. This setup isolates interference under balanced spatial partitioning, similar to ZNS-friendly workloads [14].
- (2) **Uneven Partitioning.** To model heterogeneous tenant allocations, we assign progressively larger regions to each workload, ranging from 2% to 23% of device capacity in 3% increments, summing to 100%. This creates non-uniform utilization and stresses the SSD’s ability to preserve isolation under heterogeneous spatial utilization.

Results. Figure 4 shows WAF, latency, and throughput. Under even partitioning, both FDP and Conv exhibit an ideal WAF of ~ 1 and similar latency and IOPS. Since logical regions are uniformly sized and distributed, writes spread uniformly across erase blocks at similar rates, resulting in minimal internal SSD data movement. In contrast, clear divergence appears under uneven partitioning. While Conv WAF increases to ~ 3 , FDP maintains a WAF of ~ 1 . Here, invalidation rates (due to wrap-around overwrites) diverge across concurrent write streams, leading to uneven invalidation within erase blocks in Conv. With FDP RUH hinting, streams with similar invalidation rates are isolated, preserving behavior similar to even partitioning. In this setting, FDP reduces average latency by 70% and tail latencies (P99–P99.99) by approximately 77% by mitigating GC interference, while improving IOPS by 3.3×. *Uneven partitioning is common in production systems, where multi-tenant workloads exhibit heterogeneous data footprints and growth rates.*

Takeaway 1: FDP provides strong isolation of concurrent sequential writers under both even and uneven partitioning, maintaining near-ideal WAF and reducing average and tail latencies by 70% and 77%, whereas conventional SSDs suffer from GC-induced interference.

3.2 RUH Isolation With Shared Sequential Workloads

Setup. To stress the benefits of FDP isolation when a RUH is shared by multiple sequential write workloads, we increase the number of workloads mapped to each RUH under *uneven partitioning* by subdividing each per-RUH workload into multiple non-overlapping sub-workloads and running them concurrently. Specifically, the uneven workload assigned to each RUH in § 3.1 is evenly partitioned into distinct LBA ranges while preserving the same aggregate size and access patterns. For **16 workloads**, two workloads are assigned to each RUH. This design introduces intra-RUH contention and models realistic deployments where strict one-to-one mapping between workloads and RUHs is impractical, particularly given that current devices expose a limited number of RUHs (e.g., 8). Here, we vary the total number of concurrent workloads from 8 to 64, corresponding to RUH-to-workload mappings from 1:1 to 1:8.

Results. Figure 5(a) shows device WAF as the RUH-to-workload mapping changes from 1:1 (8 workloads) to 1:8 (64 workloads). As concurrency increases, Conv exhibits steadily rising WAF due to mismatched cross-workload invalidations within erase blocks, reaching 4.60 at 64 workloads. Under FDP, sharing an RUH mixes writes from different writers with mismatched invalidation patterns, also increasing WAF. Nevertheless, FDP’s WAF remains substantially lower across all concurrency levels, peaking at 1.83 at 64 workloads. This indicates that RUH-based isolation continues to mitigate cross-workload interference even with increased sharing, and suggests that the limited number of RUHs in current FDP SSDs may not be a practical barrier to effective data separation.

Figure 5(b) shows the average write latency, which mirrors the throughput results. FDP lowers the average write latency relative to Conv at every sharing ratio, by about 70% at the 1:1 mapping (42 ms vs. 145 ms) and by roughly 47% even at the highest 1:8 sharing ratio (831 ms vs. 1577 ms). Figure 5(c) shows the corresponding write IOPS. FDP sustains a consistent throughput advantage over Conv across all concurrency levels, retaining a 1.9× improvement even at the highest RUH sharing ratio (19.9 K vs. 10.5 K IOPS at 64 workloads). This demonstrates that RUH-based isolation remains effective in reducing garbage collection interference under increased RUH sharing. We further study the impact of more sophisticated partitioning policies under shared-RUH workloads in § A.2 (supplementary material).

Takeaway 2: FDP sustains lower WAF, higher IOPS, and lower write latency even under increased RUH sharing, indicating that effective isolation and invalidation of the OP space persist even with limited RUHs among concurrent sequential writers.

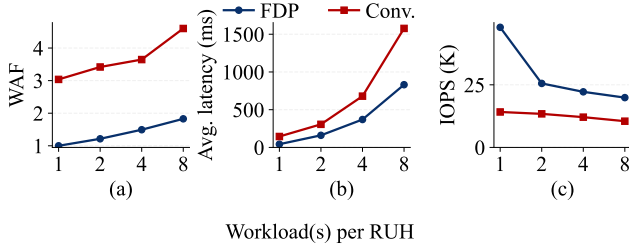


Figure 5: FDP vs. Conv SSD WAF, write latency, and IOPS with sharing of RUHs.

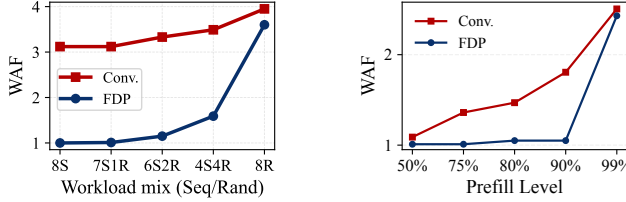


Figure 6: WAF vs. mix.

Figure 7: WAF vs. prefill ratio.

3.3 RUH Isolation With Mixed Random and Sequential Workloads

Setup. To stress SSD garbage collection and pressure on internal SSD OP space, we study the impact of random writes on FDP isolation under the (*uneven partitioning*) setup. We run eight FIO write workloads mapped to uneven logical address ranges as in § 3.1. As before, in FDP mode, each workload is pinned to a distinct RUH. We begin with eight fully sequential workloads (**8S**) and gradually introduce random writes in the workloads within its assigned range. We incrementally convert the remaining sequential stream with the smallest assigned logical size to random writes, while keeping the other streams unchanged. This process continues until all eight streams are random (**8R**). The intermediate configuration (**4S4R**) corresponds to the four smallest regions issuing random writes and the four largest remaining sequential. This setup models realistic multi-tenant workloads, where smaller or short-lived tenants often exhibit bursty random access patterns, while larger tenants retain more sequential behavior [10, 44].

Results. Figure 6 shows the results on device WAF. As in § 3.1, FDP achieves near-ideal WAF (~ 1) for fully sequential workloads (**8S**), whereas Conv incurs a WAF of ~ 3 . As randomness is progressively introduced in smaller logical regions, WAF increases for both modes. At **4S4R**, where cumulatively 26% of the LBA range is written randomly, FDP maintains a substantially lower WAF (~ 1.59) compared to Conv (~ 3.49), a 54% reduction. The effectiveness of FDP decreases as random writes introduce random invalidations across erase blocks, and the overprovisioned (OP) space cannot absorb incoming writes without triggering garbage collection and internal data movement. When all streams become random (**8R**), spatial locality largely disappears, and both modes exhibit elevated WAF (3.60 vs. 3.95), reducing FDP’s advantage to about 9%. These results show that FDP remains effective at isolating data streams as long as pressure on the shared OP space is limited. Under heavy GC pressure from random write workloads, shared OP space can lead to noisy neighbor interference [44].

Takeaway 3: FDP exhibits perfect isolation and reduces WAF under concurrent sequential and random writes as long as the invalidations of the random writes can mostly be absorbed by the internal SSD OP space.

3.4 RUH Isolation With Mixed Hot and Cold Data

Setup. In many storage systems, hot (frequently updated) and cold (rarely updated) data coexist in SSDs due to application-driven access patterns [10, 26, 30, 44]. This effect is amplified in cloud environments running multi-tenant applications, where VMs exhibit heterogeneous activity levels. To evaluate FDP’s ability to *separate* hot and cold data under challenging random write workloads, we run a single random write workload while varying the fraction of the SSD LBA range it accesses. The active region models hot data, while the remaining capacity represents cold data. We precondition the SSD by writing it to full capacity multiple times before each experiment. The *prefill ratio* denotes the fraction of the total SSD capacity written during preconditioning to simulate cold data. The random-write workload then accesses the remaining $(1 - p)$ portion of the LBA space, representing the hot region. A higher prefill ratio leaves less space for hot writes, intensifying the GC pressure on the active region. In the FDP mode, the active workload is mapped to a dedicated RUH, isolating hot writes from the cold prefill region.

Results. Figure 7 shows WAF with varying prefill ratios. With FDP, WAF remains nearly constant (around 1.0) up to 90% prefill and increases only to 2.43 at 99% utilization. Without FDP, WAF increases steadily with utilization, from 1.09 (50%) to 1.81 (90%), reaching 2.51 at 99%. At high utilization, FDP reduces WAF by up to 44%. This improved resilience is due to FDP’s ability to colocate hot LBAs within erase blocks, enabling more efficient invalidation. Together with the utilization of device OP space, this allows random writes to be absorbed more effectively.

Takeaway 4: FDP enables higher device utilization with low WAF under hot and cold data layouts by allowing the SSD internal OP space to be effectively used by hot data.

3.5 RUH Isolation Under Skewed Accesses

Setup. To evaluate how data locality affects device WAF and whether FDP mitigates interference under mixed workloads with different skew levels, we run experiments with two FIO write workloads, one accessing the LBA range uniformly at random and the other following a Zipfian distribution. We vary the Zipfian skew ($skew \in 1.0, 1.2, 1.4$) and the LBA allocation between the workloads using splits of 20/80, 50/50, and 80/20. Real world workloads often exhibit a mix of skewed and uniform access patterns due to heterogeneous application behavior, making it important to evaluate such combinations [49].

Results. To set up a baseline for Zipfian access patterns, we ran a single workload with varying values of the Zipfian constant. Consistent with previous SSD WAF results with varying skew [21, 24], our results show a strong inverse relationship between skew and WAF due to repeated accesses to LBAs resulting in lower pressure on OP space. With uniform random writes ($skew = 0$), WAF rises rapidly and saturates near 4.0. In contrast, highly skewed workloads

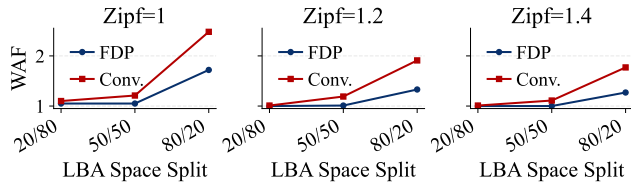


Figure 8: FDP vs. Conv. SSD WAF under mixed workloads with varying Zipf skew values and LBA space splits (uniform/Zipfian).

(e.g., skew = 1.4) sustain WAF close to 1.0. Full characterization of this baseline is provided in § A.1 (supplementary material).

Figure 8 shows the impact on the mixed write workload (1 random and 1 Zipfian) setup across different splits and skew values. In both Conv and FDP modes, device WAF increases as the fraction of LBA space allocated to the random workload grows, due to higher pressure on OP space from widely distributed invalidations across the LBA range. However, FDP is more resilient as it confines random and Zipfian workloads to separate LBA ranges, resulting in higher invalidation locality and lower pressure on OP space. As skew increases, FDP’s benefit becomes more pronounced. For example, at skew = 1.4 with the 80/20 split, FDP maintains a WAF of 1.25, while Conv degrades to 1.77.

Takeaway 5: FDP improves WAF under mixed workloads with skews by preserving locality, with gains increasing as skew grows and random access pressure on the LBA range rises.

More Takeaways. Our microbenchmarks also show scalable WAF behavior across all three FDP device sizes and demonstrate effective WAF isolation for mixed workloads using filesystem temperature hints across XFS, EXT4, Btrfs, and F2FS in § A.4 (supplementary material). We omit these results from the main paper due to space constraints.

3.6 FDP Data Placement Guidelines

- 1. Group writes by invalidation rate.** Placement groups should contain data that becomes invalid at similar rates due to spatial and temporal locality. Under uneven sequential workloads, separating streams by RUH reduces WAF from approximately 3 to 1 because each RUH contains writes with a similar invalidation rate.
- 2. Share RUHs only among compatible write streams.** A one-to-one mapping between writers and RUHs is not necessary. FDP maintains lower WAF and higher throughput when up to eight sequential writers share each RUH. However, WAF increases from approximately 1 to 1.83 as the sharing ratio increases from 1:1 to 1:8. RUHs should therefore be shared among streams with similar access and invalidation patterns.
- 3. Preserve locality for random writes.** FDP reduces interference between sequential and random streams when random invalidations remain localized to maximize the use of the internal SSD erase blocks. With four sequential and four random streams, FDP reduces WAF from 3.49 to 1.59.
- 4. Separate hot and cold data.** Placing hot random writes in a separate RUH prevents movement of cold data during garbage collection and ensures their invalidations are localized. FDP maintains a WAF near 1 through 90% prefill, compared with 1.81 without FDP.
- 5. Separate streams with different access skew.** Uniform and skewed random workloads should use different placement groups.

Levels	WAL	L0~3	L4	>=L5
Write Hints	SHORT	MEDIUM	LONG	EXTREME

Table 2: VFS write hint assignment for RocksDB.

Skewed workloads repeatedly invalidate a smaller set of LBAs, while uniform workloads distribute invalidations across a larger region.

6. Use the minimum number of placement groups that preserves distinct invalidation behavior. Assigning every file, tenant, or writer to a separate RUH is unnecessary and may exceed the limited number of RUHs exposed by the current generation of devices. Placement policies should first classify streams by invalidation rate, locality, working-set size and write rate, and then merge classes with similar behavior.

4 ROCKSDB DATA PLACEMENT

In this section, we describe the data placement strategy used in RocksDB leveraging its LSM-tree-based architecture that only performs out of place writes on the storage layer. Based on our analysis of the distinct lifetimes of the WAL and SST levels (§ 2.3.1) and the isolation of concurrent sequential writes (§ 3.1), we use four RUHs to separate the WAL, L4, L5 and higher, and L0–L3. We implement this data placement design through two storage paths:

1. Through the filesystem using *experimental* VFS patches that propagate lifetime hints. (§ 4.1)
2. Through the raw character device using the *stable* io_uring_cmd interface (§ 4.2).

4.1 Data Placement Using Filesystem Hints

We modify RocksDB to exploit its architectural data separation through VFS write-lifetime hints. The WAL and SST levels already form distinct write streams with different expected lifetimes and write volumes, but this information is lost at the filesystem interface. When RocksDB creates a WAL or SST file, it sets a lifetime hint using `F_SET_FILE_RW_HINT`. RocksDB supports configurable data placement using the `fcntl()` system call. Its internal write lifetime hint enumeration (`WriteLifetimeHint`) mirrors the Linux VFS enum `rw_hint` values of `RWH_WRITE_LIFE_{SHORT, MEDIUM, LONG, EXTREME}`.

We use the smallest number of lifetime classes supported by our architectural and empirical analysis (§ 2.3.1). The WAL is assigned `WLTH_SHORT`, L0–L3 share `WLTH_MEDIUM`, L4 uses `WLTH_LONG`, and L5 and deeper levels share `WLTH_EXTREME`. This policy uses four hints rather than assigning a separate hint to every level. It separates components only where their expected lifetimes differ substantially and avoids consuming additional RUHs. During flush and compaction, each output SST file receives the hint associated with its destination level. The SSD can therefore place writes with similar expected invalidation times in the same reclaim units, reducing data movement during garbage collection. *The RocksDB integration requires about 25 lines of code changes.*

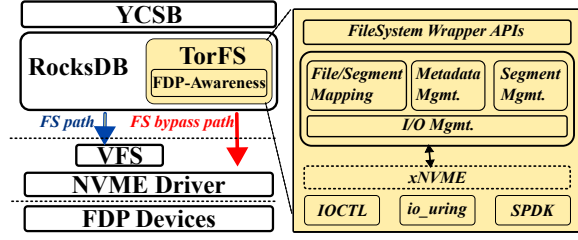


Figure 9: Overview of FS and FS bypass on RocksDB.

4.2 Data Placement Using I/O Passthru (Filesystem Bypass) - TorFS

RocksDB provides abstract wrapper APIs for pluggable storage backends. To use FDP data placement with only stable Linux kernel interfaces, we developed TorFS (Figure 9). TorFS implements the RocksDB Filesystem API and uses xNVM [48] to issue NVMe commands from user space through the `io_uring_cmd` interface utilizing the raw character device. It applies the data placement policy described in Table 2. Instead of passing lifetime hints through the filesystem, TorFS directly specifies the placement identifier (PID) associated with the target RUH in each NVMe write command. This path bypasses the VFS, filesystem, and block layers while retaining the NVMe driver in the Linux kernel. TorFS consists of approximately 4,000 lines of C++ code. It provides a control path for evaluating whether bypassing the filesystem shows any performance advantage to control device WAF via application-controlled file to LBA mapping while preserving the same FDP placement policy and device WAF.

FDP-Aware Data Placement. TorFS intercepts `WriteLifeTimeHint` of RocksDB and maps each file to one of the five FDP Reclaim Unit Handles (0–4). WAL files are mapped to RUH 0, L0–L3 SST files to RUH 1, L4 SST files to RUH 2, and L5+ SST files to RUH 3. It additionally maps its MANIFEST files to RUH 4. This mapping groups data with similar lifetimes into physically isolated Reclaim Units.

How Files Map Region on Raw Device. TorFS maintains a global mapping table to track the mapping between files and segments. Each segment defined by an offset and length, represents a range of LBAs. TorFS allocates one or more segments based on the file size and records the segments in the mapping table after files are flushed to the device, and uses the mapping table to quickly locate the segments and retrieve the data.

Read and Write I/O Paths. Each file pre-allocates a DMA-pinned write buffer upon creation via the `xnvm_buf_alloc` API, avoiding repeated allocations on the critical path. Incoming writes are accumulated in this buffer. When the buffer is full or `Sync` is invoked, `DataSync()` aligns the payload to block boundaries, splits it according to the device’s maximum transfer size, and submits batched NVMe write commands tagged with the file’s Placement Handle. TorFS issues aligned NVMe read commands via the `io_uring_cmd` I/O Passthru interface, bypassing the kernel page cache and eliminating kernel-to-user copies incurred by the VFS path.

Metadata Management. Metadata operations (e.g., file creation, update, rename, and deletion) are recorded as tagged, append-only entries in a dedicated MetaSegment. Each entry includes a typed header (Base, Update, Replace, Delete), avoiding in-place journaling overhead. When the log region of the MetaSegment reaches

a threshold, a background thread asynchronously merges the accumulated incremental operations into a compact baseline snapshot and writes it back to the main metadata region. This compaction process does not block foreground write operations.

Space Management and Deallocation. TorFS mitigates post-deletion fragmentation using a priority-queue-based segment allocator. Free LBA ranges are maintained in a min-heap ordered by offset, enabling on-the-fly merging of contiguous segments and preserving allocation contiguity. Upon file deletion, segments are enqueued into a shared `deallocate_queue` and processed by background worker threads. Workers merge adjacent segments and issue batched NVMe Dataset Management (DSM) Deallocate commands, informing the SSD of invalidated LBAs and reducing GC pressure.

To prevent overwhelming the SSD under delete-heavy workloads, TorFS employs a queue-depth-aware throttling policy with thresholds T_{low} and T_{high} ($T_{low} < T_{high}$). We use the following heuristic where Q denotes the de-allocate queue depth:

If $Q < T_{low}$, all segments are reclaimed immediately.

If $T_{low} \leq Q < T_{high}$, only segments larger than 64 KB are reclaimed.

If $Q \geq T_{high}$, only segments larger than 1 MB are reclaimed.

This policy ensures that high delete pressure does not flood the SSD controller with fine-grained Deallocate commands.

5 MYSQL DATA PLACEMENT

We modify MySQL to expose the architectural separation among its major write streams through VFS lifetime hints. InnoDB generates three distinct heavy write patterns due to redo-log writes, double-write buffer writes, and buffer-manager writes to user tablespaces. The redo log is written sequentially over a large cyclic region. The double-write buffer is also written sequentially but is reused more frequently because it occupies a smaller cyclic region. Buffer-manager flushes produce random, in-place writes over much larger user tablespaces.

When MySQL opens each file, it assigns a lifetime hint using `F_SET_FILE_RW_HINT`. We assign `WRITE_LIFE_SHORT` to double-write buffer files, `WRITE_LIFE_MEDIUM` to redo-log files, and `WRITE_LIFE_EXTREME` to user tablespaces written by the buffer manager. The double-write buffer receives the "short" hint because its small cyclic region is overwritten frequently. The larger redo-log region is reused less frequently and receives a "medium" hint. User tablespaces receive the "extreme" hint because their pages are distributed over a large address space and generally remain valid longer. Temporary files are set to "long" while metadata files to store schema and control information are set to "not set". The VFS hints are just used to separate data and their relative temperature is not semantically important.

Why only VFS integration? An I/O Passthru implementation similar to TorFS (§ 4.2) was impractical within the scope of this paper. RocksDB exposes a pluggable Filesystem API and primarily writes immutable SST files, making a character device practical. InnoDB instead relies on filesystem semantics for mutable user tablespaces, redo logs, temporary files, and double-write buffers. Our RocksDB evaluation (§ 6.1) already compares filesystem hinting with filesystem bypass and isolates their effects on WAF and performance.

6 EVALUATION

Our evaluation covers RocksDB and MySQL. All experiments are conducted on a single server equipped with dual Intel Xeon Gold 6342 processors (2×24 cores, 2.80 GHz), 498 GB DRAM, and a 3.84 TB FDP-capable NVMe SSD, running Linux 6.9.0. We use RocksDB 7.2.2 and Percona Server 8.0.34. For RocksDB, we compare three configurations, namely TorFS with FDP, FDP on XFS (§ 4.1), and a conventional baseline on XFS, using YCSB [17] and db_bench [23] workloads. For MySQL, we run InnoDB on ext4 using TPC-C to evaluate the data separation benefits outlined in § 5.

6.1 RocksDB

In this section, we evaluate RocksDB using multiple workloads from YCSB for macro-benchmarks simulating realistic cloud applications, and db_bench for microbenchmarks targeting specific I/O patterns used in past SSD interface work [14]. Our experiments cover single-tenant and multi-tenant settings. In the single-tenant setting, each benchmark executes independently to characterize baseline write amplification, throughput, and latency. In the multi-tenant setting, multiple instances of the workload and RocksDB are run concurrently to emulate realistic deployments with competing tenants, to assess performance isolation, resource contention, and write amplification under concurrent tenant access.

6.1.1 YCSB-A (Update-Intensive). We evaluate FDP’s performance under YCSB Workload A using the standard read-update ratio (50/50) and a highly write-intensive ratio (20/80) to stress the system, comparing FDP on XFS and TorFS against a Conv baseline. Both configurations use 40 million records, each consisting of a default YCSB key (~23 B) and a 40 KB value (10 fields of 4 KB each), totaling approximately 1.6 TB, together with 200 million operations, 6 client threads, and the standard request distribution with a Zipfian constant of 0.99. The total amount of data written to the SSD over the full experiment is approximately 94 TB.

Figure 10 shows the result of this experiment. Both FDP-XFS and TorFS reach an ideal WAF of ~1 across both ratios after a brief warm-up, while the Conv baseline rises to approximately 1.39 with noticeable spikes. The near-perfect overlap between TorFS and FDP-XFS confirms that FDP’s benefit stems from its placement isolation mechanism rather than TorFS-specific (OS bypass) optimizations. The consistent results across both read-update ratios demonstrate that FDP’s data separation effectiveness is robust to increasing update intensity and write rates under out-of-place writes (delete-append write pattern).

Figure 11 shows tail latency at 95th, 99th, 99.9th, and 99.99th percentiles. FDP variants show lower tail latencies than Conv due to less GC interference. FDP-XFS reduces p99.99 read and update latency from 44 383 μ s and 45 791 μ s (Conv) to 32 959 μ s and 35 775 μ s. TorFS achieves the maximum gain by minimizing OS layer overhead, with p99.99 read and update latencies of 30 815 μ s and 35 775 μ s, representing reductions of 31% and 22% over Conv. The results for the 20/80 workload are provided in § B.1(supplementary material) and show consistent behavior, demonstrating the effectiveness of our data separation policies against write amplification in LSM-tree-based key-value stores while preserving QoS for latency-sensitive workloads.

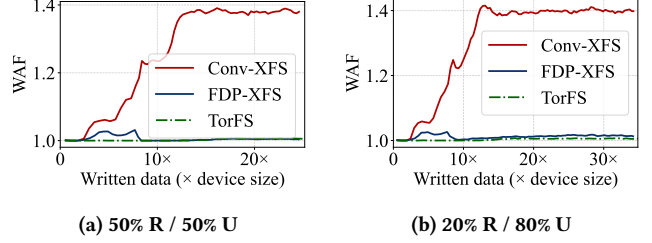


Figure 10: WAF under different read/update ratios for YCSB Workload A.

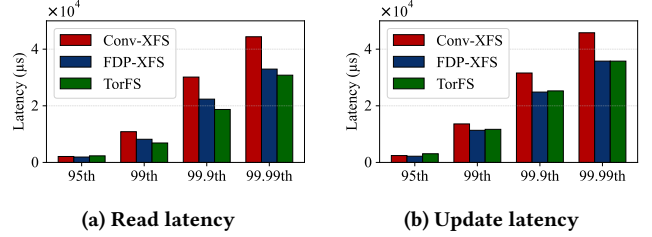


Figure 11: Tail latency percentiles for YCSB Workload A (50/50 read-update).

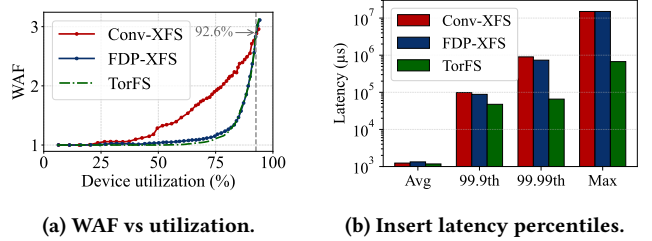


Figure 12: WAF and tail latency under YCSB Workload E.

6.1.2 Modified YCSB-E (Insert-Intensive). We evaluate the robustness of our data separation policy using YCSB Workload E consisting of 95% inserts and 5% scans, simulating data ingestion patterns typical of logging systems, time-series databases, and event streaming applications. The workload inserts 90 million records in total, occupying approximately 3.4 TB (95% of device capacity).

Figure 12a plots interval WAF against device utilization as the insert-heavy workload grows the dataset from roughly 6% to 94% of the device capacity. Conv-XFS diverges early because data mixing from SST and WAL files during compaction, as new inserts trigger compactions that repeatedly co-locate and rewrite data from different insertion phases, pushes its WAF above 1.10 at about 43% utilization and to 2.09 at 80%. Both FDP-XFS and TorFS suppress this effect substantially. FDP-XFS stays below 1.10 until roughly 70% utilization and reaches only 1.25 at 80%, while TorFS remains near 1.00 up to 60% and reaches 1.20 at 80%. The gap is widest between 70% and 85% utilization, where Conv-XFS incurs 1.6–1.7× the WAF of either FDP configuration. Beyond roughly 85%, however, all three configurations degrade sharply as the free space available to garbage collection vanishes, and the FDP curves meet Conv-XFS at about 92.6% utilization (dashed line), where all three reach a WAF of about 2.9. Placement isolation is therefore most beneficial for insert-heavy workloads while the device retains free space. As the

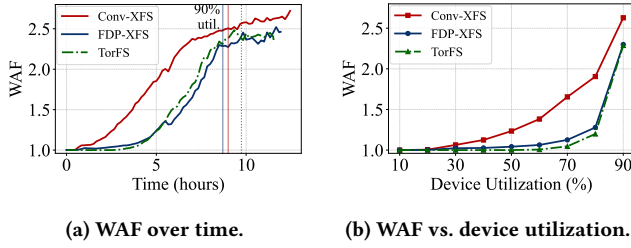


Figure 13: WAF comparison under large-scale stress test.

device approaches full capacity, GC pressure from the shortage of free blocks dominates and placement alone can no longer contain write amplification.

Figure 12b shows insert tail latency across four percentile tiers. The three approaches exhibit broadly comparable latency at p95 through p99.9. At p99.9, *TorFS* achieves dramatically lower latency than both *FDP-XFS* and *Conv-XFS* (106,623 μ s vs. 2,209,791 μ s and 2,529,279 μ s, a 20.7 $\times$ and 23.7 $\times$ reduction). This extreme-tail advantage suggests *TorFS*'s direct device access and deallocation policies reduce rare I/O queuing and GC overheads substantially, delivering superior QoS precisely where latency spikes are most disruptive to application responsiveness.

6.1.3 SSD Utilization Stress Test. To evaluate FDP's effectiveness under sustained high SSD utilization conditions, we conduct a stress test using YCSB Workload A (50/50 read-update) with 75 million records totalling approximately 3 TB. As the workload progresses, continuous updates drive device utilization above 90%, allowing us to examine how FDP's WAF benefit evolves as available SSD space nears exhaustion. We could reduce this value by increasing the compaction frequency but we retain the default setting to stress device utilization.

Figure 13 presents two complementary views of the same experiment, WAF over time and WAF at each SSD utilization level, respectively. Below 30% utilization, all three configurations maintain WAF near 1.0 with negligible differences. FDP's advantage grows steadily as utilization increases, peaking between 70% and 80%, where FDP-XFS and TorFS reduce WAF by 33% and 37% over Conv-XFS (from 1.91 to 1.28 and 1.20, respectively). This is the region in which lifetime-based data isolation is most effective with sufficient Reclaim Unit capacity remaining to sustain clean per-RU GC boundaries. Beyond 90% utilization (vertical lines at approximately 8.7, 9.0, and 10.4 experiment hours for FDP-XFS, Conv-XFS, and TorFS), the SSD OP space cannot be effectively used to buffer the writes and it is forced to co-locate data with different lifetime hints within the same Reclaim Units, and the gap narrows to 12–13% (Conv: 2.63, FDP-XFS: 2.30, TorFS: 2.29). Nevertheless, FDP consistently outperforms Conv across all utilization levels, confirming that lifetime-aware placement provides WAF benefits even at high practical SSD utilizations. The latency results from the same experiment follow a similar pattern, with p95 through p99.9 remaining broadly comparable across configurations while p99.99 shows meaningful reductions under FDP, as detailed in § B.2 (supplementary material).

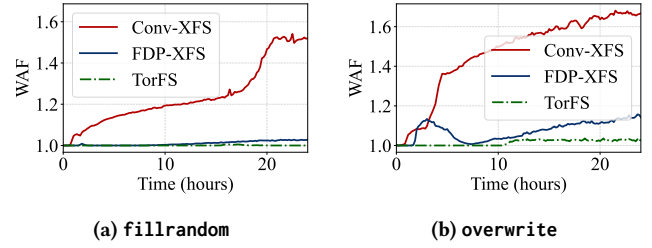


Figure 14: WAF under db_bench workloads.

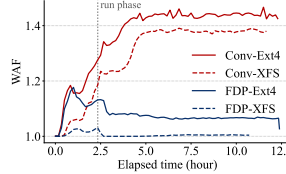


Figure 15: WAF under Conv and FDP on ext4 and XFS.

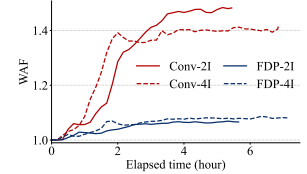


Figure 16: WAF for 2- and 4-instance RocksDB.

6.1.4 db_bench Workloads. We evaluate RocksDB's WAF under two representative db_bench workloads used in previous ZNS studies [14], *fillrandom*, which models bulk data ingestion via sequential random-key inserts, and *overwrite*, which models steady-state update pressure via repeated overwrites of existing keys. We compare FDP-enabled configurations on both XFS and TorFS against a Conv baseline on XFS. Both workloads target approximately 2 TB of device capacity. The *overwrite* workload uses a two-phase procedure: an initial *fillseq* phase rapidly populates the full key space, followed by an *overwrite* phase that drives sustained compaction pressure during the measurement period.

Under the *fillrandom* workload (Figure 14a), Conv-XFS exhibits rapid WAF escalation as the dataset grows and compaction activity intensifies, reaching approximately 1.5–1.6. FDP-XFS achieves moderate improvement by directing data to separate reclaim units, while TorFS maintains a near-flat WAF of approximately 1.05 throughout the experiment. Under the *overwrite* workload (Figure 14b), the divergence between configurations becomes more pronounced over time. Conv-XFS again escalates significantly, reflecting continuous invalidation of stale data across mixed-temperature blocks, while TorFS sustains a stable WAF near 1.05 for the duration of the workload. The advantage of TorFS over FDP-XFS across both workloads suggests that filesystem-level co-design with FDP is important to fully realizing the device's placement capabilities.

6.1.5 Filesystem Sensitivity on RocksDB. Figure 15 shows that FDP's WAF benefit on RocksDB is essentially filesystem-neutral. Both curves peak early and decay under FDP, instead of climbing monotonically as under Conv, and both settle far below their Conv baseline, reducing steady state WAF by 26.0% on ext4 (1.44 to 1.07) and 27.3% on XFS (1.38 to 1.00). The one gap that does not close is between the filesystems themselves. ext4 and XFS differ by about 0.06 WAF under both Conv and FDP, and under FDP that gap makes up most of ext4's residual overhead above the ideal value of 1 (0.066 for ext4 versus 0.005 for XFS), pointing to ext4's own allocation and journaling behaviour rather than to placement. Because XFS consistently reaches

the lower WAF of the two, we use it as the baseline filesystem for the remaining RocksDB experiments.

6.1.6 Multi-Tenant RocksDB Deployment. Multi-tenant deployments with containerized RocksDB instances represent a common pattern in modern cloud infrastructure. To evaluate FDP’s effectiveness in such scenarios, we deploy two and four containerized RocksDB instances executing concurrent YCSB Workload A (50/50 read-update) sharing the same FDP SSD while scaling the setup. Each container maps RocksDB levels to placement handles as described in Table 2 and shares the RUHs across the instances for data of similar lifetimes. Both configurations maintain a consistent aggregate dataset of approximately 1.6 TB. The two-instance setup allocates 20 million records per instance, while the four-instance setup allocates 10 million records per instance, each issuing 50 million operations.

Figure 16 shows WAF and latency results under multi-tenant workloads. FDP achieves steady-state WAF of 1.07 and 1.08 for 2- and 4-instance configurations, compared to 1.48 and 1.40 for Conv, representing 28% and 23% reductions. This improvement stems from FDP’s isolation of data of similar lifetimes in RU’s preventing unnecessary data movement. Since RUHs are shared across containers, data of similar lifetimes from different containers are mapped to the same RUH, preventing intermixing across tenants and leading to more efficient use of SSD internal OP. Average read and update latencies remain comparable across both configurations, indicating that FDP’s placement mechanism does not introduce measurable throughput overhead under multi-tenant workloads. These results highlight the benefit of sharing RUHs across tenants with the same placement policy. Although the number of RUHs on an SSD is limited and tenants must share them rather than each owning a dedicated set, FDP remains effective across multi-tenant configurations.

6.2 MySQL

We evaluate FDP on Percona Server 8.0.34 with InnoDB on ext4 using tpcc-mysql’s TPC-C workload [39] with 128 concurrent client connections to sustain SSD write pressure. Each experiment comprises a *data loading phase* that populates the warehouses, followed by a *transaction-processing phase* that executes five transaction types (New-Order, Payment, Order-Status, Delivery, and Stock-Level) [34].

6.2.1 File Classification Strategy. This section evaluates how assigning different Reclaim Unit Handles (RUHs) to different file types affects WAF. Based on the write patterns and relative write volumes of each file category described in § 5, we define five placement policies and assess their WAF impact under a TPC-C workload of 6000 warehouses. WAF values are sampled at multiple time points during the final hour of each run and averaged to obtain a representative steady-state metric. The five policies and their resulting average WAF are summarised in Table 3.

Table 3 yields three observations. First, Policies 1–3 all achieve a WAF of 1.24 at the reported precision, indicating that assigning log, temporary table, and metadata files to additional RUHs provides no observable WAF benefit. This result is consistent with their similar lifetimes and small share of the total write volume. Second, Policy 4 reduces WAF from 1.80 to 1.53 (15.0%) relative to Policy 5

Table 3: File placement policies and average WAF.

Policy	User Table	DW Buffer	Log	Temp	Meta	WAF
1	EXT	SHT	MED	LNG	NS	1.24
2	EXT	SHT	MED	MED	NS	1.24
3	EXT	SHT	NS	NS	NS	1.24
4	EXT	NS	NS	NS	NS	1.53
5 (Conv.)	NS	NS	NS	NS	NS	1.80

DW Buffer = Double-Write Buffer. EXT = WLTH_EXTREME, SHT = WLTH_SHORT, MED = WLTH_MEDIUM, LNG = WLTH_LONG, NS = WLTH_NOT_SET.

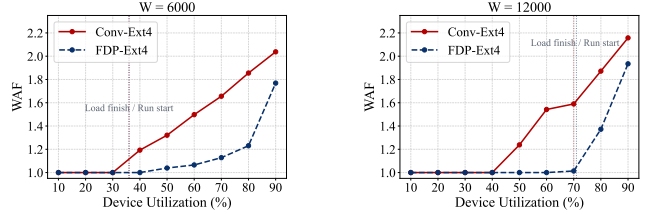


Figure 17: WAF as device utilization increases during TPC-C runs with 6000 (left) and 12000 (right) warehouses.

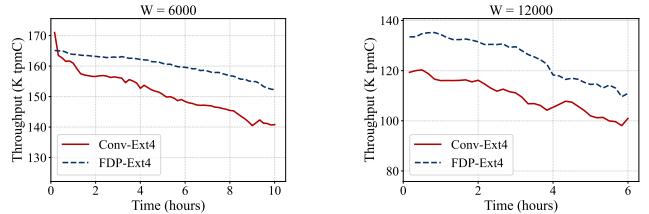


Figure 18: TPC-C throughput over time for Conv and FDP with 6000 (left) and 12000 (right) warehouses.

(Conv) by isolating user table files, suggesting that separating long-lived user data mitigates GC interference among files with different lifetimes. Third, Policy 3 further reduces WAF from 1.53 to 1.24 (19.0%) relative to Policy 4 by separating double-write buffer files from user table files. This additional improvement is consistent with the high write volume and frequent overwrites of double-write buffers. Policy 3 achieves the same 31.1% WAF reduction relative to Conv as Policies 1 and 2 while using only three RUHs, the fewest among these policies. It maps user table files to EXTREME, double-write buffer files to SHORT, and all remaining files to the default NOT_SET. We therefore adopt Policy 3 as the baseline placement policy for all subsequent experiments.

6.2.2 Scalability Across SSD Utilization. Building on the minimal-RUH policy established above, we further evaluate FDP’s effectiveness as SSD utilization increases within a single sustained workload. We run TPC-C under both Conv and FDP at two warehouse counts, 6000 (15 hours) and 12000 (6 hours), letting device utilization climb from near-empty toward 90% over the course of each run, and report WAF as a function of the utilization level reached.

Figure 17 shows WAF as device utilization increases at $W = 6000$ and $W = 12000$. At $W = 6000$, both configurations maintain a WAF of approximately 1.0 through 30% utilization. At 40%, Conv’s WAF rises to 1.19, while FDP’s remains at 1.00. FDP’s relative WAF reduction peaks at approximately 34% at 80% utilization (1.23 versus 1.86), then declines to 13% at 90% (1.77 versus 2.04). At $W = 12000$,

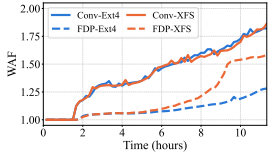


Figure 19: WAF on ext4 and xfs.

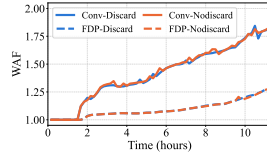


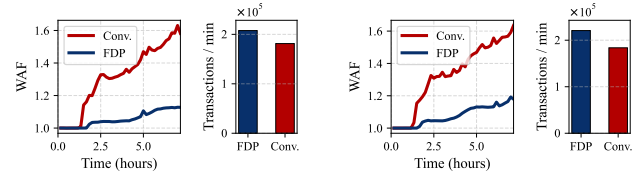
Figure 20: WAF with vs. without discard.

Conv’s WAF begins to rise at 50% utilization, while FDP’s remains approximately 1.0 through 70%. The relative reduction peaks at approximately 36% at 70% utilization (1.01 versus 1.59), then declines to 10% at 90% (1.94 versus 2.16). Comparisons across database sizes should account for the different LOAD-phase boundaries (approximately 36% utilization at $W = 6000$ and 70% at $W = 12000$), as the same utilization may correspond to different workload phases. Both database sizes exhibit a non-monotonic trend, with FDP’s relative WAF reduction increasing initially and diminishing near saturation. Figure 18 shows that these WAF reductions are accompanied by higher throughput. FDP improves average TpmC by 9.7% at $W = 6000$ (158 757 versus 144 736) and 13.7% at $W = 12000$ (124 998 versus 109 895).

6.2.3 Filesystem Sensitivity on MySQL. Figure 19 shows that ext4 and XFS exhibit similar WAF under Conv throughout the run (median relative difference $\approx 1\%$, with occasional spikes up to $\approx 3.6\%$), ending at 1.82 and 1.86, respectively. Under FDP, the two curves remain close for approximately the first 6 hours (~ 24 TiB of cumulative host writes from the start of the LOAD phase, equivalent to $\sim 7\times$ the device capacity), after which the gap widens. At approximately 7.7 hours (~ 32 TiB of cumulative host writes, 62–64% device utilization), XFS’s WAF is 6.1% higher than ext4’s. It subsequently increases more rapidly, while ext4’s WAF rises more gradually. By the end of the run, FDP’s interval WAF reaches 1.28 on ext4 and 1.59 on XFS, with ext4’s WAF 19.3% lower than XFS’s. Relative to the corresponding Conv baselines, these values represent WAF reductions of 29.6% on ext4 and 14.9% on XFS. Thus, FDP provides a larger WAF reduction on ext4 in this experiment, with the advantage emerging primarily during the later, higher-utilization portion of the run. We therefore use ext4 as the baseline filesystem for the MySQL experiments.

6.2.4 Discard is Not a Confounding Variable. Figure 20 evaluates the effect of the `discard` mount option by comparing ext4 with and without discard under both Conv and FDP. The two Conv curves closely track each other, with a median relative difference of 0.5% and a maximum transient difference of 3.4%. The FDP curves are even closer, with a median relative difference of 0.1%. For each placement mode, the final WAF values differ by less than 2% between discard settings, reaching approximately 1.82 for Conv and 1.28 for FDP. FDP reduces WAF relative to Conv by 29.3% with discard and 29.6% without it. These results indicate that the discard mount option has little effect on WAF under this workload and does not materially affect FDP’s benefit on ext4.

6.2.5 Multi-Tenant MySQL Deployment. In multi-tenant scenarios, each MySQL instance adopts the placement Policy 3 described in Table 3. Files in the same placement class share an RUH across



(a) Two-instance workload

(b) Four-instance workload

Figure 21: WAF under multi-tenant MySQL workloads. FDP achieves 12–16% WAF reduction across both configurations.

instances, preserving separation between file classes without requiring additional RUHs for each tenant. We evaluate two and four tenants with 3000 and 1500 warehouses per instance, respectively, both totaling 6000 warehouses and reaching $\sim 70\%$ SSD utilization representative of production workloads.

Figure 21 shows that FDP reduces WAF from 1.58 to 1.13 with two tenants (28.7%) and from 1.64 to 1.18 with four tenants (27.9%) by the end of the run. The similar reductions indicate that FDP retains its benefit as the tenant count increases from two to four at a fixed aggregate database size. Conv’s WAF exceeds 1.10 at approximately 1.5 hours in both configurations, whereas FDP crosses the same threshold at approximately 5 hours with two tenants and 4.3 hours with four tenants. This delay of approximately 2.8–3.5 hours suggests that separating file classes helps limit WAF growth as device utilization rises. These WAF reductions are accompanied by higher average host write bandwidth during the 6-hour RUN phase. With two tenants, bandwidth increases from 1517 MB/s under Conv to 1709 MB/s under FDP (12.7%). With four tenants, it increases from 1531 MB/s to 1837 MB/s (20.0%).

The MySQL experiments demonstrate that FDP reduces device-level WAF with only a few placement streams for InnoDB’s mix of B-tree page updates and log writes. Together with the RocksDB results, these findings support the applicability of lifetime-based placement across LSM-tree and B-tree storage architectures.

7 CONCLUSION

NVMe FDP is an industry-driven shift in the SSD interface landscape whose backward compatibility and ability to reduce WAF with non-invasive software changes is driving adoption in data-center deployments. With stable Linux support for I/O Passthru and emerging filesystem-based lifetime hints, FDP enables practical data separation in widely deployed storage systems. We show that RocksDB’s out-of-place architecture can be leveraged for WAF gains to separate SST and WAL files with different lifetimes using both filesystem and I/O Passthru interfaces. Similar gains can also be obtained in MySQL’s predominantly in-place update architecture by separating data from logs, table data, and metadata using filesystem hints. Across synthetic and real-world workloads, we show that FDP delivers near-ideal WAF in RocksDB and achieves a 10–15% WAF reduction in MySQL at high SSD utilization under both single-tenant and multi-tenant settings.

REFERENCES

- [1] 2024. Amazon Sustainability. <https://sustainability.aboutamazon.com/climate-solutions>.
- [2] 2024. Climate change is humanity's next big moonshot. <https://blog.google/outrreach-initiatives/sustainability/dear-earth/>.
- [3] 2024. Our Path to Net Zero. <https://sustainability.fb.com/wp-content/uploads/2023/07/Meta-2023-Path-to-Net-Zero.pdf>.
- [4] 2024. [PATCH] nvme: enable FDP support - Kanchan Joshi. Available at <https://lore.kernel.org/linux-nvme/20240510134015.29717-1-joshi.k@samsung.com/>.
- [5] 2024. Re: [PATCH] ext4: Pass write-hint for buffered IO - Theodore Ts'o. Available at <https://lore.kernel.org/all/173099237654.321265.6588244483471280365.b4-ty@mit.edu/T/>.
- [6] 2024. TP4146 Flexible Data Placement Ratified Technical Proposal. Available under ratified TPs at <https://nvmexpress.org/specification/nvm-express-base-specification>.
- [7] 2025. FDP file I/O - Kanchan Joshi. Available at <https://lore.kernel.org/all/20250729145135.12463-1-joshi.k@samsung.com/>.
- [8] 2025. Flexible data placement. Available at <https://lwn.net/Articles/1018642/>.
- [9] Nitin Agrawal, Vijayan Prabhakaran, Ted Wobber, John D. Davis, Mark S. Manasse, and Rina Panigrahy. 2008. Design Tradeoffs for SSD Performance. In *2008 USENIX Annual Technical Conference, Boston, MA, USA, June 22-27, 2008. Proceedings*, Rebecca Isaacs and Yuan Yuan Zhou (Eds.). USENIX Association, 57–70. https://www.usenix.org/legacy/event/usenix08/tech/full_papers/agrawal/agrawal.pdf
- [10] Michael Allison, Arun George, Javier González, Dan Helmick, Vikash Kumar, Roshan R. Nair, and Vivek Shah. 2025. Towards Efficient Flash Caches with Emerging NVMe Flexible Data Placement SSDs. In *Proceedings of the Twentieth European Conference on Computer Systems, EuroSys 2025, Rotterdam, The Netherlands, 30 March 2025 - 3 April 2025*. ACM, 1142–1160. <https://doi.org/10.1145/3689031.3696091>
- [11] Michael Allison and John Rudelic. 2024. What Is the NVMe Express Flexible Data Placement (FDP)? Storage Developer Conference 2023, YouTube video. https://www.youtube.com/watch?v=c8Jfw_WANn6A Accessed: September 29, 2026.
- [12] Jens Axboe. 2024. fio: Flexible I/O Tester. <https://github.com/axboe/fio>. Accessed: 2026-03-28.
- [13] Benjamin Berg, Daniel S. Berger, Sara McAllister, Isaac Grosf, Sathya Gunasekar, Jimmy Lu, Michael Uhlar, Jim Carrig, Nathan Beckmann, Mor Harchol-Balter, and Gregory R. Ganger. 2020. The CacheLib Caching Engine: Design and Experiences at Scale. In *14th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2020, Virtual Event, November 4-6, 2020*. USENIX Association, 753–768. <https://www.usenix.org/conference/osdi20/presentation/berg>
- [14] Matias Björling, Abutalib Aghayev, Hans Holmberg, Aravind Ramesh, Damien Le Moal, Gregory R. Ganger, and George Amvrosiadis. 2021. ZNS: Avoiding the Block Interface Tax for Flash-based SSDs. In *Proceedings of the 2021 USENIX Annual Technical Conference, USENIX ATC 2021, July 14-16, 2021, Irina Calciu and Geoff Kuenning (Eds.)*. USENIX Association, 689–703. <https://www.usenix.org/conference/atc21/presentation/bjorling>
- [15] Matias Björling, Javier González, and Philippe Bonnet. 2017. LightNVM: The Linux Open-Channel SSD Subsystem. In *15th USENIX Conference on File and Storage Technologies, FAST 2017, Santa Clara, CA, USA, February 27 - March 2, 2017*, Geoff Kuenning and Carl A. Waldspurger (Eds.). USENIX Association, 359–374. <https://www.usenix.org/conference/fast17/technical-sessions/presentation/bjorling>
- [16] Silvia Botros and Jeremy Tinley. 2021. *High Performance MySQL* (4 ed.). O'Reilly.
- [17] Brian F. Cooper, Adam Silberstein, Erwin Tam, Raghu Ramakrishnan, and Russell Sears. 2010. Benchmarking cloud serving systems with YCSB. In *Proceedings of the 1st ACM Symposium on Cloud Computing* (Indianapolis, Indiana, USA) (SoCC '10). Association for Computing Machinery, New York, NY, USA, 143–154. <https://doi.org/10.1145/1807128.1807152>
- [18] Oracle Corporation. 2024. InnoDB Architecture. <https://dev.mysql.com/doc/refman/8.0/en/innodb-architecture.html>. Accessed: 2026-03-25.
- [19] DapuStor Corporation. 2024. Flexible Data Placement: The Next Level of SSD Solutions. <https://en.dapustor.com/news/111.html>
- [20] DapuStor Corporation. 2026. DapuStor NVMe FDP Deep Dive: From Concept to Real-World Deployment. <https://en.dapustor.com/news/142.html>
- [21] Peter Desnoyers. 2014. Analytic Models of SSD Write Performance. *ACM Trans. Storage* 10, 2 (2014), 8:1–8:25. <https://doi.org/10.1145/2577384>
- [22] Siying Dong, Andrew Kryczka, Yanqin Jin, and Michael Stumm. 2021. RocksDB: Evolution of Development Priorities in a Key-value Store Serving Large-scale Applications. *ACM Trans. Storage* 17, 4 (2021), 26:1–26:32. <https://doi.org/10.1145/3483840>
- [23] Facebook. 2013. RocksDB: A Persistent Key-Value Store for Flash and RAM Storage. <https://github.com/facebook/rocksdb>.
- [24] Gabriel Haas, Bohyun Lee, Philippe Bonnet, and Viktor Leis. 2025. SSD-iq: Uncovering the Hidden Side of SSD Performance. *Proc. VLDB Endow.* 18, 11 (2025), 4295–4308. <https://doi.org/10.14778/3749646.3749694>
- [25] Joseph M. Hellerstein, Michael Stonebraker, and James R. Hamilton. 2007. Architecture of a Database System. *Found. Trends Databases* 1, 2 (2007), 141–259. <https://doi.org/10.1561/19000000002>
- [26] Jian Huang, Anirudh Badam, Laura Caulfield, Suman Nath, Sudipta Sengupta, Bikash Sharma, and Moinuddin K. Qureshi. 2017. FlashBlox: Achieving Both Performance Isolation and Uniform Lifetime for Virtualized SSDs. In *15th USENIX Conference on File and Storage Technologies, FAST 2017, Santa Clara, CA, USA, February 27 - March 2, 2017*, Geoff Kuenning and Carl A. Waldspurger (Eds.). USENIX Association, 375–390. <https://www.usenix.org/conference/fast17/technical-sessions/presentation/huang>
- [27] Percona Inc. 2023. Percona Server for MySQL. <https://www.percona.com/software/mysql-database/percona-server> Accessed: 2026-03-25.
- [28] Hyunjoon Jin, Fanny Potkin, Wen-Yee Lee, Anton Bridge, and Max A. Cherney. 2025. The AI Frenzy Is Driving a New Global Supply Chain Crisis. *Reuters* (3 December 2025). <https://www.reuters.com/world/china/ai-frenzy-is-driving-new-global-supply-chain-crisis-2025-12-03/> Accessed: 12 March 2026.
- [29] Kanchan Joshi, Anuj Gupta, Javier González, Ankit Kumar, Krishna Kanth Reddy, Arun George, Simon Andreas Frimann Lund, and Jens Axboe. 2024. I/O Passthru: Upstreaming a flexible and efficient I/O Path in Linux. In *22nd USENIX Conference on File and Storage Technologies, FAST 2024, Santa Clara, CA, USA, February 27-29, 2024*, Xiaosong Ma and Youjip Won (Eds.). USENIX Association, 107–121. <https://www.usenix.org/conference/fast24/presentation/joshi>
- [30] Kang, Jeong-Uk and Hyun, Jeeseok and Maeng, Hyunjoon and Cho, Sangyeun. 2014. The Multi-streamed Solid-State Drive. In *6th USENIX Workshop on Hot Topics in Storage and File Systems (HotStorage 14)*.
- [31] KIOXIA. [n.d.]. UFROP: Userland I/O Filesystem for RocksDB with FDP SSD. <https://github.com/kioxia-jp/ufrop>. Accessed: 2026-08-01.
- [32] KIOXIA America, Inc. 2025. How FDP Can Reduce Garbage Collection, and Ultimately WAF! Use FDP-enabled KIOXIA XD8 Series Data Center SSDs. <https://blog-us.kioxia.com/post/2025/06/20/how-fdp-can-reduce-garbage-collection-and-ultimately-waf-use-fdp-enabled-kioxia-xd8-series-data-center-ssds>
- [33] Bohyun Lee, Tobias Ziegler, and Viktor Leis. 2026. How to Write to SSDs. arXiv:2603.09927 [cs.DB] <https://arxiv.org/abs/2603.09927>
- [34] Scott T Leutenegger and Daniel Dias. 1993. A modeling study of the TPC-C benchmark. *ACM Sigmod Record* 22, 2 (1993), 22–31.
- [35] Huaicheng Li, Martin L. Putra, Ronald Shi, Fadhil I. Kurnia, Xing Lin, Jaeyoung Do, Achmad Imam Kistijantoro, Gregory R. Ganger, and Haryadi S. Gunawi. 2023. Extending and Programming the NVMe I/O Deterministic Interface for Flash Arrays. *ACM Trans. Storage* 19, 1 (2023), 5:1–5:33. <https://doi.org/10.1145/3568427>
- [36] Chen Luo and Michael J. Carey. 2018. LSM-based Storage Techniques: A Survey. *CoRR* abs/1812.07527 (2018). arXiv:1812.07527 <http://arxiv.org/abs/1812.07527>
- [37] Melanie Nagakawa. 2024. On the road to 2030: Our 2022 environmental sustainability report. <https://blogs.microsoft.com/on-the-issues/2023/05/10/2022-environmental-sustainability-report/>.
- [38] Patrick E. O'Neil, Edward Cheng, Dieter Gawlick, and Elizabeth J. O'Neil. 1996. The Log-Structured Merge-Tree (LSM-Tree). *Acta Informatica* 33, 4 (1996), 351–385. <https://doi.org/10.1007/S002360050048>
- [39] Percona Lab. [n.d.]. tpcc-mysql. <https://github.com/Percona-Lab/tpcc-mysql>. Accessed: 2026-09-30.
- [40] Chris Sabol and Smriti Desai. [n.d.]. SmartFTL SSDs. <https://www.youtube.com/watch?v=303zDrpt3uM> ([n.d.]).
- [41] Samsung Electronics. 2026. Samsung's PM9D3a Solid State Drive. <https://semiconductor.samsung.com/news-events/tech-blog/samsung-pm9d3a-solid-state-drive/>. Accessed: September 2026.
- [42] Samsung Semiconductor. 2024. Flexible Data Placement. <https://semiconductor.samsung.com/news-events/tech-blog/flexible-data-placement/>
- [43] Samsung Semiconductor. 2025. NVMe FDP: A Promising New SSD Data Placement Approach. <https://semiconductor.samsung.com/news-events/tech-blog/nvme-fdp-a-promising-new-ssd-data-placement-approach/>. Accessed: 2026-03-29.
- [44] Inho Song, Shoaib Asif Qazi, Javier González, Matias Björling, Sam H. Noh, and Huaicheng Li. 2026. Characterizing and Emulating FDP SSDs with WARP. In *24th USENIX Conference on File and Storage Technologies, FAST 2026, Santa Clara, CA, USA, February 24-26, 2026*, André Brinkmann and Philip Shilane (Eds.). USENIX Association, 347–362. <https://www.usenix.org/conference/fast26/presentation/song>
- [45] Swamit Tannu and Prashant J. Nair. 2023. The Dirty Secret of SSDs: Embodied Carbon. *SIGENERGY Energy Inform. Rev.* 3, 3 (oct 2023), 4–9. <https://doi.org/10.1145/3630614.3630616>
- [46] TrendForce. 2025. Enterprise SSD Prices and Shipments Surge in 3Q25, Industry Revenue Climbs 28%. *TrendForce Press Center* (5 December 2025). <https://www.trendforce.com/presscenter/news/20251205-12819.html> Accessed: 12 March 2026.
- [47] Suzhen Wu, Sijie Lan, Jindong Zhou, Hong Jiang, and Zhirong Shen. 2020. BitFlip: A bit-flipping scheme for reducing read latency and improving reliability of flash memory. In *Proc. IEEE MSST*.

- [48] xNVMe Project. 2024. xNVMe: Cross-platform NVMe user-space library. <https://github.com/xnvme/xnvme>.
- [49] Yue Yang and Jianwen Zhu. 2016. Write Skew and Zipf Distribution: Evidence and Implications. *ACM Trans. Storage* 12, 4 (2016), 21:1–21:19. <https://doi.org/10.1145/2908557>
- [50] Xiangqun Zhang, Janki Bhimani, Shuyi Pei, Eunji Lee, Sungjin Lee, Yoon Jae Seong, Eui Jin Kim, Changho Choi, Eyee Hyun Nam, Jongmoo Choi, and Bryan S. Kim. 2025. Storage Abstractions for SSDs: The Past, Present, and Future. *ACM Trans. Storage* 21, 1 (2025), 2:1–2:44. <https://doi.org/10.1145/3708992>

A THE PROMISE OF THE RAW FDP DEVICE (CONTD.)

A.1 Skew vs WAF

Figure 22 shows WAF as a function of the total amount of data written by the user (TiB) for a single workload of Zipfian random writes at eight skew levels ($skew \in \{0, 0.2, 0.4, 0.6, 0.8, 1.0, 1.2, 1.4\}$).

At low skew values ($skew \leq 0.6$), writes are distributed nearly uniformly across the LBA space, causing WAF to rise steeply and saturate at approximately 4.2 after 15–20 TiB of user writes (approximately 4–5× the device capacity). This is consistent with the behaviour of a nearly full device under uniform random writes. As skew increases ($skew \in \{0.8, 1.0\}$), writes concentrate on a smaller hot region. The resulting locality reduces the fraction of cold data intermixed with hot writes during GC, and WAF saturates at progressively lower levels of approximately 1.6 and 1.2, respectively. At high skew ($skew \geq 1.2$), writes are highly concentrated, and WAF remains close to 1.0 throughout the entire write range, indicating that GC operates almost exclusively on invalidated hot pages with minimal migration of valid pages.

These results establish the relationship between access skew and device WAF under a single workload. When natural locality is insufficient (i.e., at low skew), FDP can enforce physical separation between hot and cold data streams to achieve comparable WAF reductions, as evaluated in the mixed workload experiments in § 3.5.

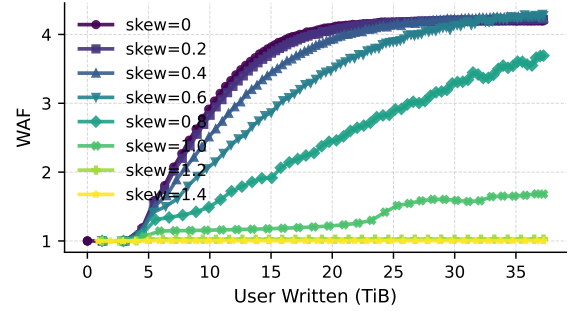


Figure 22: Baseline WAF evolution over time (User Written TiB) for a single workload under varying Zipfian skewness parameters. Higher skewness (e.g., $skew = 1.4$) maintains near-ideal WAF, while uniform random workloads ($skew = 0$) result in high write amplification.

A.2 Homogeneous vs. Heterogeneous RUH Placement

Setup. This experiment builds directly on § 3.1 and retains the same workload construction and logical layout. Each logical region is subdivided into workloads of equal size that run in parallel, preserving the aggregate write volume and access characteristics. We evaluate four concurrency levels (24, 32, 48, and 64 jobs) to examine how sensitivity to RUH placement changes with increasing parallelism.

We compare three configurations.

- **Conv** uses a conventional namespace without FDP support and serves as the baseline.

- **U-FDP (Homogeneous Placement)** enables FDP with uniform placement, assigning each RUH write streams with equally sized LBA ranges.
- **H-FDP (Heterogeneous Placement)** enables FDP with heterogeneous placement, assigning each RUH write streams with maximally diverse LBA range sizes. The ratio of the largest to the smallest range within a RUH reaches 11.5×

All three configurations use identical logical offsets, total write volumes, and access patterns. U-FDP and H-FDP use the same LBA ranges for each job. They differ only in the `fdp_pli` assignment, which determines the RUH to which each write stream is directed.

Results. Figure 23 summarizes the impact of RUH placement under increasing concurrency (24–64 jobs).

Across all levels of parallelism, U-FDP consistently maintains lower write amplification and more stable tail latency than both H-FDP and the conventional baseline (Conv). However, sensitivity to RUH assignment increases markedly with concurrency.

At moderate concurrency (WL24–WL32), H-FDP still provides benefits over Conv, although differences in LBA range sizes within each RUH already increase write amplification. By WL48, this advantage effectively disappears. H-FDP exhibits write amplification and tail latency nearly indistinguishable from the baseline, indicating that improper stream colocation largely negates FDP’s isolation benefits at this scale.

The trend persists at WL64, confirming that under extreme parallelism, RUH mapping quality becomes the dominant factor determining whether FDP isolation remains effective. In contrast, homogeneous placement continues to limit WAF growth and maintain predictable latency across all concurrency levels.

Discussion. Two consistent trends emerge across the concurrency levels. First, RUH placement quality directly affects write amplification. Homogeneous placement systematically reduces imbalance within each RUH and preserves FDP’s isolation benefits. Second, sensitivity to placement increases with parallelism. The performance gap between homogeneous and heterogeneous placement widens from WL24 to WL64. Under extreme concurrency, improper stream colocation largely nullifies FDP’s advantage.

Takeaway 6: *FDP provides strong isolation benefits, but its effectiveness increasingly depends on RUH assignment quality as concurrency grows. Grouping streams with equally sized LBA ranges within a RUH preserves scalability, whereas heterogeneous colocation produces behavior increasingly similar to that of conventional SSDs under high parallelism.*

A.3 Scalability of FDP Across Device Sizes

Experimental setup. We repeat the uneven sequential fio benchmark from Section 3.1 on three NVMe devices with capacities of 1.88 TB, 3.84 TB, and 7.52 TB, varying the number of jobs per RUH across 2, 4, 6, and 8.

Results. Figure 24 reports WAF at steady state for U-FDP and Conv across all three devices.

On the 3.84 TB device, Conv WAF rises from 2.99 to 3.97 as concurrency increases, a 33% increase driven by interference between streams. U-FDP mitigates this pressure through explicit placement, reducing WAF by 49% at peak load. On the 1.88 TB device, Conv

WAF remains largely stable across all concurrency levels (2.99–3.00), while U-FDP consistently reduces WAF by 40%. This indicates that placement isolation remains beneficial even when concurrency does not substantially increase baseline WAF.

The 7.52 TB device exhibits the lowest absolute WAF under both configurations. Conv WAF rises modestly from 2.43 to 2.79, while U-FDP maintains WAF at approximately 1.0–1.13 throughout, yielding a 59% reduction at peak load. The larger address space limits the magnitude of interference but does not eliminate the advantage of explicit placement control.

Takeaway 7: *U-FDP consistently reduces WAF across all device sizes and concurrency levels, with reductions of 40% to 59% at peak load. Larger devices limit the magnitude of interference through their larger physical address space, but explicit placement control remains effective across capacities. These results suggest that FDP is broadly applicable, including in configurations without capacity constraints.*

Contract. The physical address space available to each stream governs the magnitude of interference but does not replace explicit placement control. U-FDP consistently reduces WAF across the full range of capacities tested.

A.4 Multiple Containers with Similar Filesystem Workloads

Write Type	Block Size (KB)	File Size (MB)	Write Hint	# Files
Sequential	128	100	Short	1200
Sequential	64	64	Medium	1800
Random	8	24	Long	4000
Random	4	24	extreme	2000

Table 4: FIO Write Type Configurations for FileSystem

Setup. We evaluate how FDP interacts with filesystem allocation policies under workloads with mixed data lifetimes. We modify the Linux kernel to propagate `write_hint` values associated with files or inodes to Placement Identifiers (PLIs) at the device level, enabling placement based on data lifetime without altering filesystem internals.

All experiments use FIO with `io_uring` and direct I/O. To eliminate layout bias, we first precondition the SSD with large sequential writes to regions of 2.0 TB and 1.3 TB, followed by three hours of random overwrites with mixed write sizes to establish fragmentation at steady state. We then run a workload for eight hours with four concurrent lifetime classes, namely *short* (128 KB sequential), *medium* (64 KB sequential), *long* (8 KB sequential), and *extreme* (4 KB random). Each class writes to independent directories and is tagged with a distinct `write_hint`, allowing FDP to separate data by lifetime during placement. We run the same workload on EXT4, XFS, Btrfs, and F2FS to isolate the effects of each filesystem.

Results. Write amplification. Figure 25 shows that FDP consistently reduces WAF across all filesystems. Under the conventional SSD configuration (Conv), WAF increases to approximately 1.8 as the device fills, reflecting interference between data with different lifetimes within erase blocks. In contrast, FDP configurations stabilize near 1.0, indicating that physical write volume nearly matches logical write volume even with heterogeneous update granularities.

Performance. Figure 26 presents latency and throughput results. EXT4, Btrfs, and F2FS show moderate improvements under FDP,

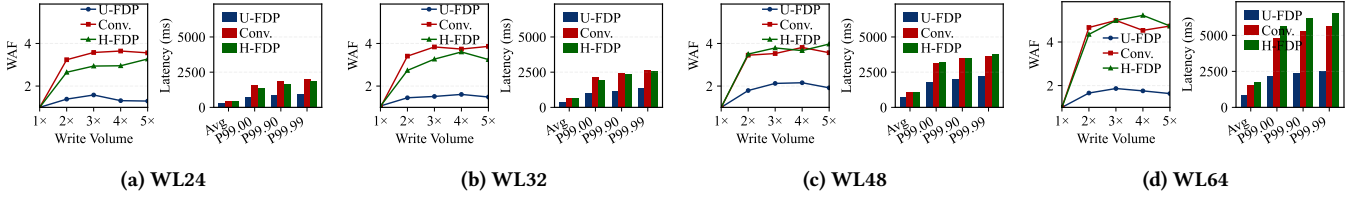


Figure 23: Impact of RUH placement under increasing concurrency. The left panels show write amplification versus cumulative write volume normalized to device capacity. The right panels show average and tail latencies through P99.99. Homogeneous placement (U-FDP) consistently maintains lower amplification and stable latency. As concurrency increases, heterogeneous placement produces behavior progressively closer to that of conventional SSDs.

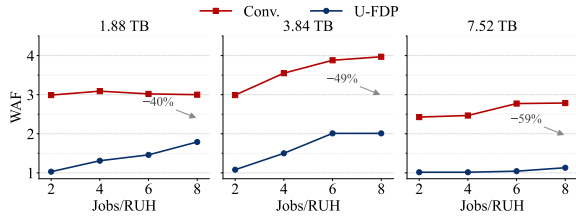


Figure 24: Steady-state WAF under U-FDP and Conv as a function of per-RUH job count across three devices of increasing capacity. Annotations indicate WAF reduction at maximum concurrency (8 jobs/RUH).

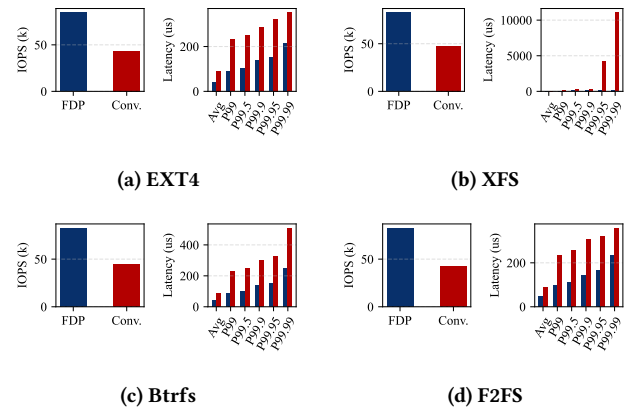


Figure 26: Comparison of latency and IOPS across filesystems.

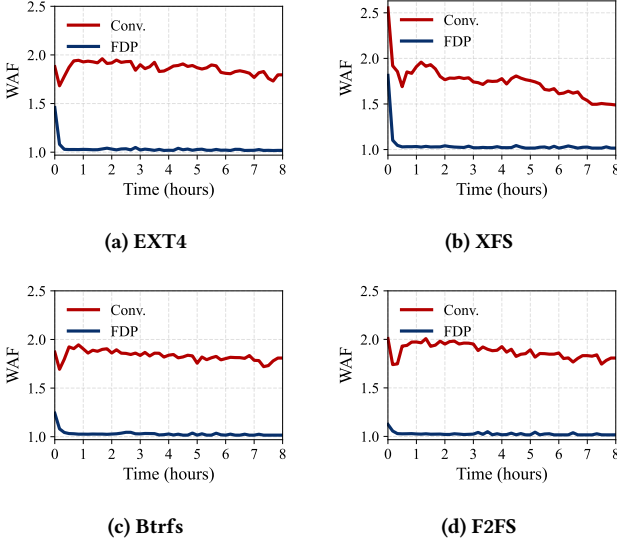


Figure 25: Comparison of WAF across four Linux filesystems with and without FDP. FDP consistently reduces write amplification across all filesystems.

with P99.99 latency reduced by 34–51% while maintaining similar IOPS. XFS shows substantially larger gains, achieving approximately 83 K IOPS compared with 47 K under Conv, along with consistently lower tail latency across percentiles. We attribute this improvement to the allocation group architecture of XFS, which aligns naturally with FDP’s placement model based on streams and better preserves spatial isolation across data classes.

Takeaway 8: Across diverse filesystems, FDP consistently restores efficient placement based on data lifetime under mixed workloads.

Conventional SSDs suffer from interference between data with different lifetimes, increasing WAF and tail latency. FDP isolates these heterogeneous streams during placement, stabilizing WAF near 1.0. The gains are particularly pronounced when filesystem allocation structures, such as XFS allocation groups, align with FDP’s placement granularity. This highlights the importance of compatibility between filesystem layout and stream isolation at the device level.

B ROCKSDB EVALUATION (CONTD.)

B.1 Update-Intensive Workloads with Different read-update Ratio

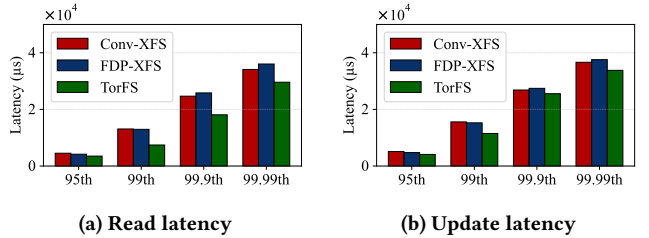


Figure 27: Tail latency percentiles for 20% read / 80% update workload. Under high update pressure, FDP maintains stable tail latencies comparable to baseline performance.

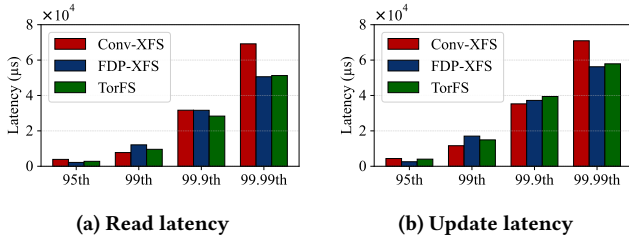


Figure 28: Tail latency under large-scale stress test.

Figures 27a and 27b show the tail latency distributions for read and update operations, respectively, under the 20/80 workload dominated by updates. Consistent with the 50/50 results in § 6.1.1, all three configurations (Conv-XFS, FDP-XFS, and TorFS) exhibit comparable latency at all measured percentiles (95th, 99th, 99.9th, and 99.99th) for both operation types. FDP reduces WAF from 1.45 to 1.05 under this workload without introducing an observable latency penalty. These results confirm that FDP’s placement isolation mechanism does not introduce queuing delays or scheduling artifacts even as update intensity increases, supporting its applicability to workloads sensitive to latency.

B.2 Tail Latency Under the SSD Utilization Stress Test

Figure 28 shows the tail latency distributions for READ and UPDATE operations under YCSB Workload A at high device utilization. All three approaches exhibit broadly comparable latency at p95, p99, and p99.9 for both operation types.

At p99.99, FDP-XFS and TorFS reduce READ latency relative to Conv-XFS by 26.9% and 25.9%, respectively (50 591 μs and 51 263 μs versus 69 183 μs). They also reduce UPDATE latency by 20.7% and 18.4%, respectively (56,223 μs and 57 887 μs versus 70 911 μs). These improvements at the extreme tail stem from FDP’s cleaner data placement within each RU, which reduces the frequency and severity of I/O stalls caused by GC. Such stalls dominate the worst observed latencies at high device utilization. For deployments sensitive to latency where meeting p99.99 SLA requirements is critical, FDP provides a meaningful advantage even as available SSD space nears exhaustion.

C TORFS VS UFROP

Traditional file systems adopt a layered architecture in which data traverses the VFS, the file system, and the block layer before reaching the device. This separation provides generality and portability but introduces software stack overhead that can become a bottleneck when performance is critical. Storage systems designed around direct application access instead bypass the kernel file system entirely, allowing applications to write using native device semantics and eliminating this redundant processing.

Recent work applies this approach to exploit FDP semantics in RocksDB. UFROP [31] and TorFS are both RocksDB plugins for FDP SSDs that share two design principles. They use hints from RocksDB to separate data into multiple streams and bypass the kernel file system to issue direct I/O. They differ in how they lay out files on the device. UFROP partitions the user data region into blocks of a fixed size and maps each file through direct and indirect blocks. A file

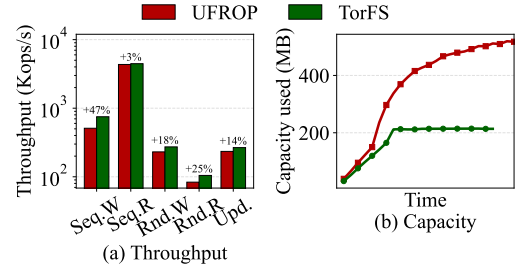


Figure 29: TorFS versus UFROP on db_bench: (a) throughput per workload and (b) on-storage space consumed over time. Percentages are relative to UFROP.

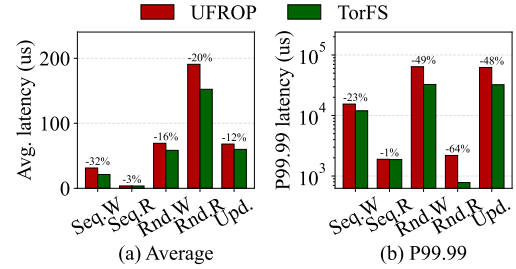


Figure 30: Latency of TorFS and UFROP on db_bench: (a) average and (b) P99.99. Percentages are relative to UFROP.

that fits within one block is stored contiguously, while larger files use indirect blocks. Unused space within a block cannot be shared with another file, resulting in internal fragmentation. The bounded number of indirect blocks per file also limits the maximum file size. TorFS instead allocates segments of variable length to match the actual file size, eliminating internal fragmentation without imposing an intrinsic limit on file size.

Performance. We compare the two plugins using db_bench with 20 million records per thread across 16 threads under sequential read, sequential write, random read, random write, and update workloads. Figure 29(a) shows nearly identical sequential read performance, with TorFS achieving a 3% improvement, since neither plugin lies on the critical path for reads. TorFS consistently outperforms UFROP under workloads dominated by writes. Sequential write throughput increases by 47%, while average and P99 latency decrease by 32% and 34%, respectively. Update throughput increases by 14%, with a 52% reduction in P99 latency (Figure 30). Random write and read throughput improve by 18% and 25%, respectively, while extreme tail latency falls by roughly half for random writes and updates. These gains stem from TorFS’s I/O submission strategy. UFROP submits requests individually, whereas TorFS batches them, reducing the number of submissions to the device and the associated overhead per request.

Space efficiency. Under the same workload, UFROP consumes 518 MB of physical space, compared with only 214 MB for TorFS, a factor of 2.4× (Figure 29(b)). TorFS’s space consumption stabilizes once the LSM tree reaches steady state, whereas UFROP’s continues to grow as fragmented blocks accumulate. This difference directly reflects the elimination of internal fragmentation through allocation based on segments. Together, these performance and space efficiency results motivate our choice of TorFS as the storage plugin.